

Branch Specialization Checkpoint

Bottlenecked Branch Routing

Branch Specialization Project

2026-05-22

Checkpoint reason: branch bottlenecks did not make unlabeled routing discover causal modularity.

Prior result:

- entropy and load balancing changed gate statistics;
- they did not reliably induce role-aligned branch modularity.

This checkpoint asks:

If each branch is capacity-bottlenecked, does cohabitation become costly enough for unlabeled routing pressure to separate local and induction roles?

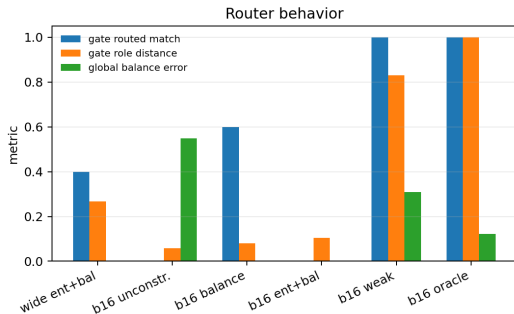
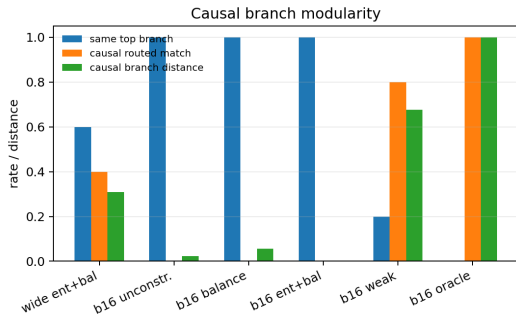
- **Same top branch:** local and induction ablations hurt most when the same branch is removed.
- **Routed role match:** local's top causal branch is branch 0 and induction's top causal branch is branch 1.
- **Branch distance:** total-variation distance between the local and induction causal branch distributions.
- **Gate routed match:** the router probabilities point to branch 0 for local positions and branch 1 for induction positions.

Same local-vs-induction competition task:

- two branch towers, shared embedding and unembedding;
- branch attention head dims: [16] instead of [64];
- five seeds per condition, 1200 training steps;
- causal modularity measured by branch ablation, not by gate statistics alone.

Conditions: unconstrained learned token router, balance-only, entropy+balance, weak-label positive control, and oracle-route upper bound.

Main Evidence



Bottlenecked unlabeled routers still co-locate local and induction causally. Weak labels mostly work; oracle routing works perfectly.

Bottlenecked Results

Condition	Local acc.	Ind. acc.	Same top	Routed match	Branch dist.
unconstrained	1.0000	0.9987	1.00	0.00	0.0220
balance only	1.0000	0.9997	1.00	0.00	0.0551
entropy+balance	1.0000	0.9982	1.00	0.00	0.0004
weak label	0.9999	0.9991	0.20	0.80	0.6768
oracle route	0.9999	1.0000	0.00	1.00	1.0000

All conditions solve the task, so this is not a performance failure.

Comparison to Wide Branches

Condition	Routed match	Branch dist.
wide64 entropy+balance	0.40	0.3082
bottleneck16 entropy+balance	0.00	0.0004
wide64 weak label	1.00	0.8957
bottleneck16 weak label	0.80	0.6768
bottleneck16 oracle route	1.00	1.0000

Shrinking branch capacity did not strengthen unlabeled modularity; if anything, the entropy+balance condition became cleaner co-location.

Supported:

- 16-dim branches can support clean modularity when routing is correct.
- Weak role-informative routing pressure still mostly induces modularity.

Not supported:

- a simple branch-capacity bottleneck makes unlabeled entropy/balance regularizers discover functional modularity.

The bottleneck is not the missing ingredient for this toy setup.

Do not spend the next iteration on more entropy/balance weights for the same task.

Next decisive experiment:

Build a conflict-heavy local-vs-induction task where cohabitation is directly costly, then retest unlabeled routing against weak-label and oracle controls.

Candidate variants: anti-correlated role labels, ambiguous shared-token layouts, or annealed weak labels that are removed after routing forms.

Sources and Provenance

- Report: `doc/phase3_toy_bottleneck_router.md`
- Model script: `scripts/toy_branch_isolation_intervention.py`
- Analysis script: `scripts/analyze_bottleneck_router_experiment.py`
- Analysis output: `results/phase3_toy_bottleneck16_router_analysis/`
- Deck data snapshot: `presentations/2026-05-22-1609-bottleneck-router/data/`
- Representative command: `python3 -u scripts/toy_branch_isolation_intervention.py --branch-head-dims 16`